CMR TECHNICAL CAMPUS
An UGC Autonomous Institute
Accredited by NBA& NAAC with A Grade
Approved by AICTE, New Delhi and Affiliated to JNTU, Hyderabad
DEPT. OF COMPUTER SCIENCE & ENGINEERING

ESTD: 2009

COMPUTER NETWORKS LAB MANUAL

Regulation : R22

Branch : B.TECH - CSE

Year & Semester : III Year/I Semester

INDEX

S.No	Name of the Experiment	Page No.
1.	Implement the data link layer framing methods such as character, character-stuffing and bitstuffing	1-6
2.	Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRCCCIP	7-10
3.	Develop a simple data link layer that performs flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.	11-14
4.	Implement Dijsktra's algorithm to compute the shortest path through a network	15-18
5.	Take an example subnet of hosts and obtain a broadcast tree for the subnet.	19-24
6.	Implement distance vector routing algorithm for obtaining routing tables at each node	25-28
7.	Implement data encryption and data decryption	29-31
8.	Write a program for congestion control using Leaky bucket algorithm.	22-41
9.	Write a program for frame sorting techniques used in buffers.	42-44
10.	How to run N map scan.	45-52
11.	Operating System Detection using N map.	53-56
12.	Study of basic Network configuration commands and utilities to debug the network issues.	57-62

1. Implement the data link layer framing methods such as character, character-stuffing and bitstuffing

Aim:

To implement and understand different data link layer framing methods such as character count, character stuffing, and bit stuffing for reliable data transmission.

Description:

Framing is an important concept in the Data Link Layer of computer networks. It is used to divide the continuous stream of data into manageable units called frames. These frames help in proper data transmission, synchronization, and error detection between sender and receiver. In this experiment, we implement different framing techniques such as Character Count, Character Stuffing, and Bit Stuffing using C language.

Character Count Method

In this method, the first field of the frame specifies the number of characters present in the frame. The receiver reads this count value to determine the frame boundaries.

However, if the count value gets corrupted, it may lead to loss of synchronization.

Example:

Data = HELLO

Frame = 5HELLO

Here, **5** represents the number of characters in the frame.

Disadvantage: If the count value is corrupted (e.g., 5 becomes 8), the receiver may misinterpret the frame boundary.

Character Stuffing

This method uses special characters to mark the beginning and end of a frame. If these special characters appear in the data, an escape character is added before them.

This ensures that the receiver does not confuse data with control characters.

Example:

Original Data = A DLE B

After Stuffing = DLE STX A DLE DLE B DLE ETX

Bit Stuffing

Bit stuffing is used in bit-oriented protocols. A special flag sequence is used to mark frame boundaries.

Whenever five consecutive '1's appear in the data, a '0' is inserted automatically. This prevents confusion with the flag sequence and ensures proper frame detection.

Example:

Original Data = 111110111

After Stuffing = 1111100111

Program:

a)Bit stuffing

```
#include<stdio.h>
#include<string.h>
voidmain()
{
intn,i,j,k,count;
chara[50],b[50];
printf("enterinputframe(0's&1's):\n");
gets(a);
n=strlen(a);
strcpy(b,"01111110");
i=0;
j=8;
while(i<n)
{
```

```
count=1;
if(a[i]=='1')
{
b[j]=a[i];
for(k=i+1;a[k]=='1'&&k<n&&count<5;k++)
{
j++;
b[j]=a[k];
count++;
if(count==5)
{
j++;
b[j]='0';
}
i=k;
}
}
else
{
b[j]=a[i];
}
i++;
j++;
}
b[j]='\0';
strcat(b,"01111110");
printf("afterstuffing,theframeis:\n");
```

```
puts(b);
}
```

b)Byte stuffing or Character stuffing

```
#include<stdio.h>
voidmain()
{
chardata[100],frame[100];
charflag[4]={'f','l','a','g'};
inti,n,j;
printf("entertheinputstring:\n");
gets(data);
n=strlen(data);
for(i=0;i<4;i++)
frame[i]=flag[1];
for(i=0,j=4;i<n;i++,j++)
{
if(data[i]=='f'&&data[i+1]=='l'&&data[i+2]=='a'&&data[i+3]=='g')
{
frame[j]='e';
j++;
frame[j]='s';
j++;
frame[j]='c';
j++;
n=n+3;
}
elseif(data[i]=='e'&&data[i+1]=='s'&&data[i+2]=='c')
```

```
{
frame[j]='e';
j++;
frame[j]='s';
j++;
frame[j]='c';
j++;
n=n+3;
}
frame[j]=data[i];
}
n=n+3;
n++;
for(i=0;i<4;i++,n++)
frame[n]=flag[i];
printf("stringafterstuffingis:");
puts(frame);
}
```

Output:

1. **Bit stuffing output:** enter input frame (0`s and 1`s only) 111111

   After stuffing the frame is 01111110111110101111110

2. **Character stuffing or byte stuffing output:** enter the input frame: abc

After character stuffing the frame is stx abc etx

Viva Questions:

1. What is framing?

Answer:

Framing is the process of dividing data into small units called frames so that the receiver can identify the start and end of each message.

2. What is the Character Count method?

Answer:

In this method, the first byte of the frame tells the total number of characters in the frame.

3. What is the main problem with the Character Count method?

Answer:

If the count value is corrupted, the receiver may lose synchronization and read the wrong data.

4. What is Character Stuffing?

Answer:

Character stuffing adds an extra special character (escape character) before special control characters in the data to avoid confusion.

5. What is Bit Stuffing?

Answer:

In bit stuffing, a '0' is inserted after five consecutive '1's in the data to prevent confusion with the frame delimiter.

2. Write a program to compute CRC code for the polynomials CRC-12, CRC-16 and CRCCCIP

Aim:

To write a program in C to compute CRC (Cyclic Redundancy Check) code using standard polynomials such as CRC-12, CRC-16, and CRC-CCITT.

Description:

Cyclic Redundancy Check (CRC) is an error detection technique used in computer networks to detect accidental changes in data during transmission. It treats the data as a binary polynomial and performs division using a predefined generator polynomial. The remainder obtained after division is called the **CRC code**, which is appended to the data before transmission. At the receiver side, the same division is performed to check for errors.CRC works based on **modulo-2 division**, where subtraction is replaced by XOR operation. The sender appends zeros equal to the degree of the generator polynomial to the data and performs division. The resulting remainder is added to the original data to form the transmitted frame. If the receiver gets a non-zero remainder after division, it indicates an error in transmission.

Common CRC Codes

Code-Parity check bits	Generator polynomial g(x)
CRC-12	$x^{12}+x^{11}+x^3+x^2+x+1$
CRC-16	$x^{16}+x^{15}+x^2+1$
CRC-CCITT	$x^{16}+x^{12}+x^5+1$
CRC-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

Program

```
#include<stdio.h>
#include<string.h>
voidmain()
{
inti,j,divlen,datalen;
chardata[100],div[30],temp[30],quot[100],rem[30],div1[30];
printf("enterdata:");
gets(data);
printf("enterdivisor");
gets(div);
divlen=strlen(div);
datalen=strlen(data);
strcpy(div1,div);
for(i=0;i<divlen-1;i++)
{
data[datalen+i]='0';
}
for(i=0;i<divlen;i++)
temp[i]=data[i];
for(i=0;i<datalen;i++)
{
quot[i]=temp[0];
if(quot[i]=='0')
{
for(j=0;j<divlen;j++)
div[j]='0';}
```

```
else
{
for(j=0;j<divlen;j++)
div[j]=div1[j];
}
(j=divlen-1;j>0;j--)
if(temp[j]==div[j])
rem[j-1]='0';
else
rem[j-1]='1';
rem[divlen-1]=data[i+divlen];
strcpy(temp,rem);
strcpy(rem,temp);
printf("\nquotientis:");
for(i=0;i<datalen;i++)
printf("%c",quot[i]);
printf("\nremainderis:");
for(i=0;i<divlen-1;i++)
printf("%c",rem[i]);
printf("\nfinaldatais:");
for(i=0;i<datalen;i++)
printf("%c",data[i]);
for(i=0;i<divlen-1;i++)
printf("%c",rem[i]);}
```

Output:

Enter data:1100000001111

Enter divisor:101011

Quotient is: 1111010111110

Reminder is:10010

Fial data is: 110000000111110010

Viva Questions:

1. What is CRC?

Answer:

CRC (Cyclic Redundancy Check) is an error detection technique used to detect errors in transmitted data.

2. Why do we append zeros before CRC division?

Answer:

We append (n−1) zeros to the data, where n is the length of the generator polynomial, to perform modulo-2 division and generate the CRC remainder.

3. What operation is used in CRC calculation?

Answer:

Modulo-2 division is used, which is performed using the XOR operation.

4. What is the difference between CRC-12, CRC-16, ?

Answer:

They differ in the size and generator polynomial used.

- CRC-12 → 12-bit polynomial
- CRC-16 → 16-bit polynomial

5. What is a codeword in CRC?

Answer:

A codeword is the original data plus the CRC remainder appended at the end for transmission.

3. Develop a simple data link layer that performs flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.

Aim:

To develop a simple Data Link Layer protocol that implements flow control using Sliding Window Protocol and error/loss recovery using Go-Back-N mechanism.

Description:

he Sliding Window Protocol is used in the Data Link Layer to control the flow of data between sender and receiver by allowing multiple frames to be sent before receiving acknowledgments. It improves efficiency by utilizing the network bandwidth effectively. In this experiment, Go-Back-N ARQ (Automatic Repeat Request) is used for error and loss recovery, where the sender retransmits all frames starting from the lost or damaged frame. The receiver only accepts frames in order and discards out-of-order frames. Acknowledgments are sent for correctly received frames to ensure reliability. This mechanism ensures proper synchronization, efficient transmission, and error control in communication systems.

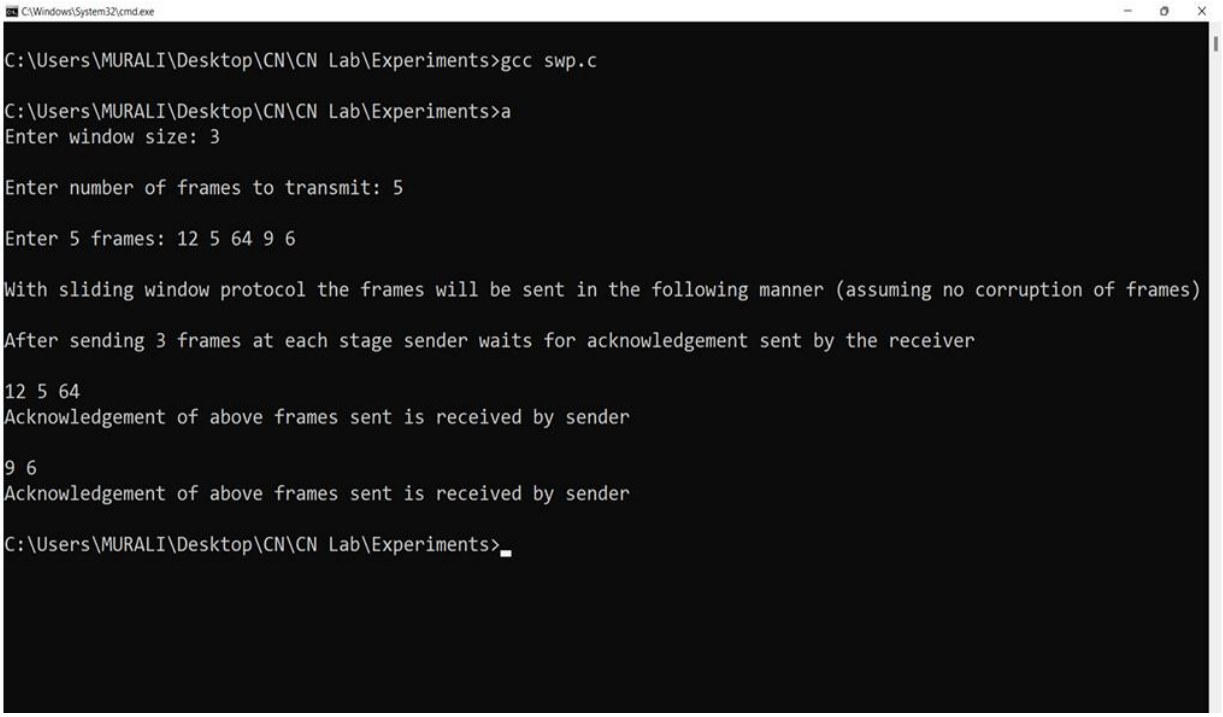
Program:

```
#include<stdio.h>
voidmain()
{
intpath[10][10],i,j,min,a[10][10],p,st=1,ed=5,stp,edp,t[10],index;
printf("Enterthecostmatrix\n");
for(i=1;i<=5;i++)
for(j=1;j<=5;j++)
scanf("%d",&a[i][j]);
printf("Enterthenumofpaths\n");
scanf("%d",&p);
printf("Enterthepossiblepaths\n");
```

```
for(i=1;i<=p;i++)
for(j=1;j<=5;j++)
scanf("%d",&path[i][j]);
for(i=1;i<=p;i++)
{
t[i]=0;
stp=st;
for(j=1;j<=5;j++)
{
edp=path[i][j+1];
t[i]=t[i]+a[stp][edp];
if(edp==ed)
break;
else
stp=edp;
}
}
min=t[st];
index=st;
for(i=1;i<=p;i++)
{
if(min>t[i])
{
min=t[i];
index=i;
}
}
```

```
printf("Minimumcost:%d\n",min);
printf("Minimumcostpath");
for(i=1;i<=5;i++)
{
printf("--->%d",path[index][i]);
if(path[index][i]==ed)
break;
}
```

}**Output:**



Viva Questions

1. What is the Sliding Window Protocol?

Answer:

Sliding Window Protocol is a flow control method where multiple frames can be sent before receiving acknowledgments, based on the window size.

2. What is Go-Back-N ARQ?

Answer:

Go-Back-N is an error control mechanism where, if a frame is lost or damaged, the sender retransmits that frame and all subsequent frames.

3. What is the purpose of window size?

Answer:

Window size determines how many frames can be sent without waiting for an acknowledgment.

4. How does the sender know a frame is lost in Go-Back-N?

Answer:

If an acknowledgment is not received within a certain time (timeout), the sender assumes the frame is lost and retransmits it.

5. What is the main advantage of Go-Back-N?

Answer:

It is simple to implement and ensures reliable data transmission.

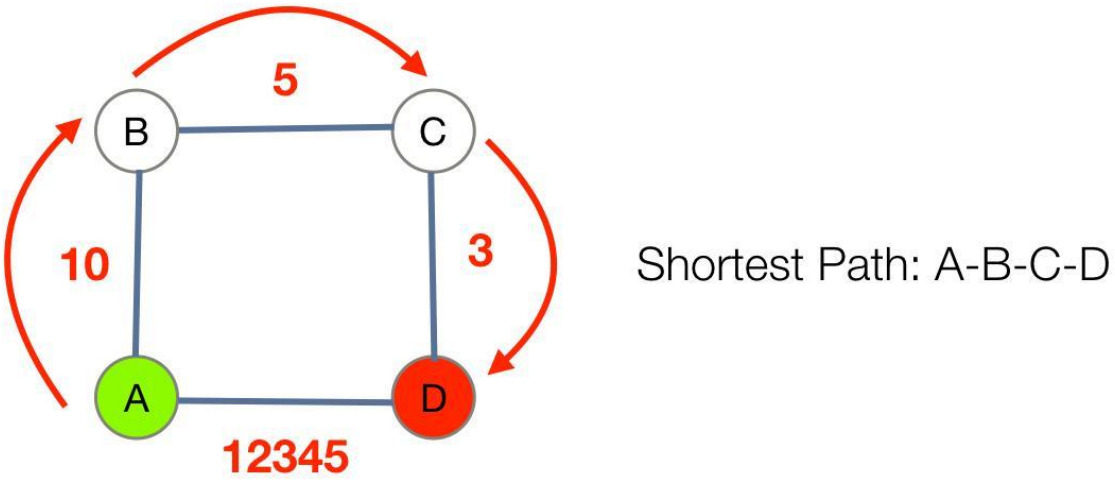
4. Implement Dijsktra's algorithm to compute the shortest path through a

network .

Aim:

To implement **Dijkstra's Algorithm** to compute the shortest path from a source node to all other nodes in a network.

Dijkstra's Algorithm is a popular greedy algorithm used in computer networks to find the shortest path between nodes in a weighted graph. It works by selecting the node with the minimum distance value and updating the distances of its neighboring nodes. Initially, the distance of the source node is set to zero, and all other nodes are set to infinity. The algorithm repeatedly picks the unvisited node with the smallest distance, marks it as visited, and relaxes its adjacent edges. This process continues until all nodes are visited. It guarantees the shortest path in graphs with non-negative edge weights. The algorithm is widely used in routing protocols and network optimization problems.



.

Program:

```
#include<stdio.h>
structnode
{
unsigneddist[20];
unsignedfrom[20];
}
rt[10];
intmain()
{
intdmat[20][20];
intn,i,j,k,count=0;
printf("\nEnterno.ofnodes:");
scanf("%d",&n);
printf("Enterthecostmatrix:\n");
for(i=0;i<n;i++)
for(j=0;j<n;j++)
{
scanf("%d",&dmat[i][j]);
dmat[i][i]=0;
rt[i].dist[j]=dmat[i][j];
rt[i].from[j]=j;
}
do
{
count=0;
for(i=0;i<n;i++)
for(j=0;j<n;j++)
```

```
for(k=0;k<n;k++)
if(rt[i].dist[j]>dmat[i][k]+rt[k].dist[j])
{
rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
rt[i].from[j]=k;
count++;
}
}
while(count!=0);
for(i=0;i<n;i++)
{
printf("\nStatevalueforrouter%dis\n",i+1);
for(j=0;j<n;j++)
{
printf("\nnode%dvia%dDistance%d",j+1,rt[i].from[j]+1,rt[i].dist[j]);
}
}
printf("\n");
}
```

Output:

```
Enter the cost matrix
0 1 4 2 0
1 0 3 7 0
4 3 0 5 0
2 7 5 0 6
0 0 0 6 0
Enter the no of paths:
```

1 2 3 4 5

1 2 4 5 0

1 3 4 5 0

1 4 5 0 0

Minimum cost :8

Minimum cost path: 1 4 5

Viva Questions:

1. What is Dijkstra's algorithm?

Answer:

Dijkstra's algorithm is a greedy algorithm used to find the shortest path from a source node to all other nodes in a weighted graph.

2. What type of graphs does Dijkstra's algorithm work on?

Answer:

It works on graphs with non-negative edge weights.

3. What is the main idea behind Dijkstra's algorithm?

Answer:

It repeatedly selects the node with the smallest tentative distance and updates the distances of its neighboring nodes.

4. Why does Dijkstra's algorithm fail for negative weights?

Answer:

Because it assumes once a shortest path is selected, it cannot be improved later, which is not true if negative weights exist.

5. What is the time complexity of Dijkstra's algorithm?

Answer:

1. Using an array: **$O(V^2)$**
2. Using a priority queue (min-heap): **$O((V + E) \log V)$**

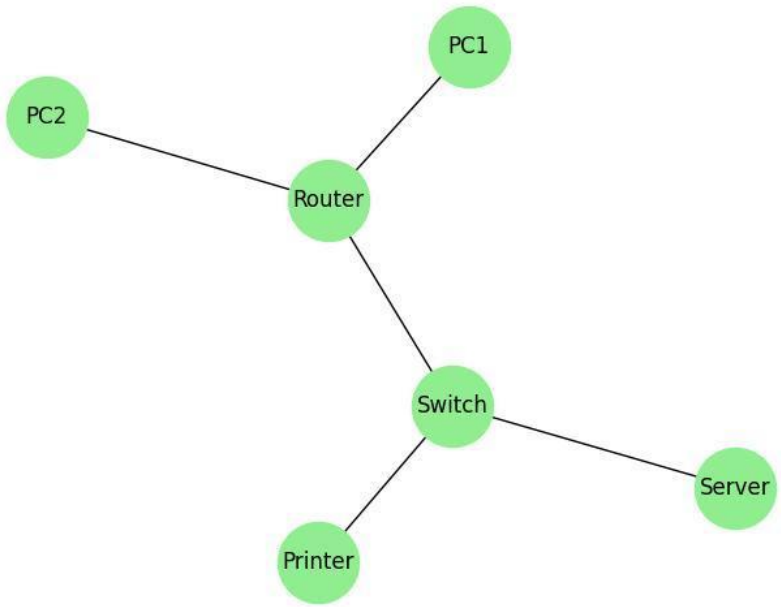
5. Take an example subnet of hosts and obtain a broadcast tree for the subnet.

Aim:

To take an example subnet of hosts and construct a **broadcast tree** for efficient data transmission.

Description:

A broadcast tree is a spanning tree used in computer networks to deliver data packets from a source node to all other nodes in a subnet without creating loops or redundant transmissions. Instead of sending multiple copies of the same message across all possible paths, the broadcast tree ensures that each node receives the message exactly once. It is derived from the network topology by selecting a root node (source) and connecting all nodes using minimum links while avoiding cycles. This improves network efficiency and reduces bandwidth usage. Broadcast trees are commonly used in routing algorithms, multicast communication, and network protocols to ensure reliable and optimized data delivery.



Program:

#include<stdio.h>

```
#include<stdlib.h>

inti,j,k,a,b,u,v,n,ne=1;

intmin,mincost=0,cost[9][9],parent[9];

intfind(int);

intuni(int,int);

voidmain()

{

printf("\nimplementationofkrushal'salgorithmforbroadcasttree\n");

printf("entertheno.ofvertices:");

scanf("%d",&n);

printf("enterthecostadjacencymatrix:\n");

for(i=1;i<=n;i++)

{

for(j=1;j<=n;j++)

{

scanf("%d",&cost[i][j]);

if(cost[i][j]==0)

cost[i][j]=99;
```

```
}
}
printf("theedgesofmin.costspanningtreeare\n");
while(ne<n)
{
for(i=1,min=99;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(cost[i][j]<min)
{
min=cost[i][j];
a=u=i;
b=v=j;
}
}
}
u=find(u);
```

```
v=find(v);
if(uni(u,v))
{
printf("%dedge(%d,%d)=%d\n",ne++,a,b,min);
mincost+=min;
}
cost[a][b]=cost[b][a]=99;
}
printf("\nmin.cost=%d\n",mincost);
}
intfind(inti)
{
while(parent[i])
i=parent[i];
returni;
}
intuni(inti,intj)
{
```

```
if(i!=j)
{
parent[j]=i;
return1;
}
return0;
}
```

Output:

```
Enter the no of vertices: 6
Enter the cost adjacency matrix
0 3 1 6 0 0
3 0 5 0 3 0
1 5 0 5 6 4
6 0 5 0 0 2
0 3 6 0 0 6
0 0 4 2 6 0
The edges of min cost spanning tree are
1 edge(1,3)=1
2 edge(4,6)=2
3 edge(1,2)=3
4 edge(2,5)=3
5 edge(3,6)=4
Min cost=13
```

Viva Questions:

1. What is a broadcast tree?

Answer:

A broadcast tree is a spanning tree used to send data from one source node to all other nodes in a network without forming loops.

2. Why do we use a broadcast tree?

Answer:

We use a broadcast tree to avoid duplicate transmissions and prevent loops in the network.

3. What is a spanning tree?

Answer:

A spanning tree is a subgraph that connects all the nodes in a network without any cycles and with minimum edges.

4. How many edges are present in a broadcast tree with n nodes?

Answer:

A broadcast tree with n nodes has (n − 1) edges.

5. What problem occurs if broadcasting is done without a tree structure?

Answer:

It may cause loops, duplicate packets, and broadcast storms in the network.

.

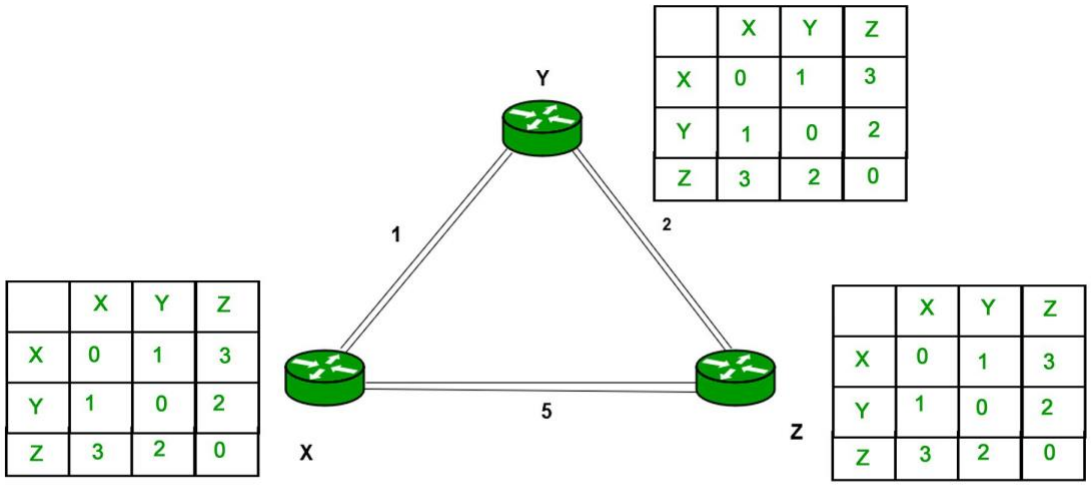
6. Implement distance vector routing algorithm for obtaining routing tables at each node

Aim:

To implement the Distance Vector Routing Algorithm for computing routing tables at each node in a network.

Description:

Distance Vector Routing is a dynamic routing algorithm used in computer networks to determine the shortest path between nodes. Each node maintains a routing table that contains the distance (cost) to reach every other node and the next hop required. Initially, nodes know only the distances to their immediate neighbors. They periodically exchange their routing tables with neighboring nodes. Based on the received information, each node updates its table using the Bellman-Ford equation, selecting the path with the minimum cost. This process continues until all nodes reach a stable state (convergence). The algorithm is simple and widely used in protocols like RIP (Routing Information Protocol)



	X	Y	Z
X	0	1	3
Y	1	0	2
Z	3	2	0

	X	Y	Z
X	0	1	3
Y	1	0	2
Z	3	2	0

	X	Y	Z
X	0	1	3
Y	1	0	2
Z	3	2	0

Program:

```
#include<stdio.h>
```

```
main()
{
chara[100],b[100],c[100];
intn,i,j;
intk=0;
printf("enter the number of elements in the array:");
scanf("%d",&n);
printf("entertheelementsto:");
for(i=0;i<n;i++)
scanf("%s",&a[i]);
printf("ceasercipheralgorithm\n");
printf("Encryption");
for(i=0;i<n;i++)
{
b[i]=a[i]+6;
printf("%c",b[i]);
}
printf("decryption");
for(i=0;i<n;i++)
{
c[i]=b[i]-6;
printf("%c",c[i]);
printf("criptanalysis\n");
for(j=1;j<25;j++)
{
for(i=0;i<n;i++,k++)
{
```

```
c[k]=b[i]-j;
printf("%c",c[k]);
printf("\n");
}
printf("\n");
}
```

Output:

```
Enter the no of nodes:4
Enter the cost matrix:
0       3       5       99
3       0       99      1
5       99      0       2
99      1       2       0
State value for router 1 is
Node 1 via 1 distance: 0
Node 2 via 2 distance: 3
Node 3 via 3 distance: 5
Node 4 via 4 distance: 4
State value for router 2 is
Node 1 via 1 distance: 3
Node 2 via 2 distance: 0
Node 3 via 3 distance: 3
Node 4 via 4 distance: 1
State value for router 3 is
Node 1 via 1 distance: 5
Node 2 via 2 distance: 3
Node 3 via 3 distance: 0
```

Node 4 via 4 distance: 2

State value for router 4 is

Node 1 via 1 distance: 4

Node 2 via 2 distance: 1

Node 3 via 3 distance: 2

Node 4 via 4 distance: 0

Viva Questions:

1. What is Distance Vector Routing?

Answer:

Distance Vector Routing is a routing algorithm where each router shares its routing table with its neighbors to find the shortest path to all nodes.

2. Which algorithm is used in Distance Vector Routing?

Answer:

It is based on the Bellman-Ford algorithm.

3. What information is stored in a routing table?

Answer:

A routing table stores destination, next hop, and cost (distance) to reach that destination.

4. What is the count-to-infinity problem?

Answer:

It is a problem where incorrect routing information causes the distance metric to increase indefinitely due to routing loops.

5. How can the count-to-infinity problem be reduced?

Answer:

It can be reduced using techniques like split horizon, route poisoning, and hold-down timer

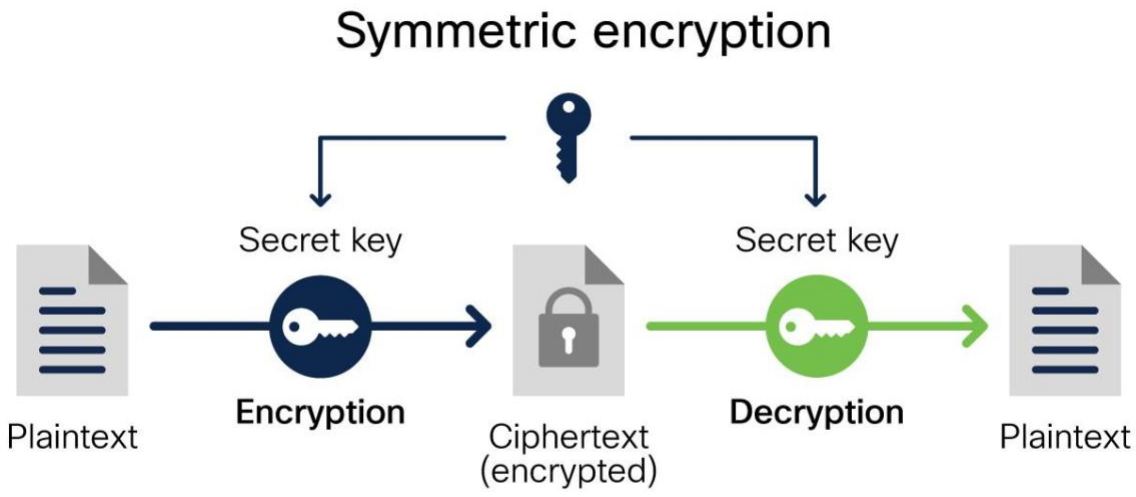
7. Implement data encryption and data decryption

Aim:

To implement data encryption and data decryption techniques to ensure secure transmission of information.

Description:

Data encryption is the process of converting plain text into an unreadable format called ciphertext using an algorithm and a key. Decryption is the reverse process, where the ciphertext is converted back into the original plain text using the same or a corresponding key. These techniques are essential for maintaining data confidentiality and security during communication. Encryption algorithms can be symmetric (same key for encryption and decryption) or asymmetric (different keys). In this experiment, a simple encryption and decryption method is implemented using C language to understand the basic working of cryptographic techniques. This ensures that unauthorized users cannot access sensitive data.



Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
voidmain()
{
inti,kl;
charcipher[50],plain[50];
charkey[50];
printf("\nenterplaintext:");
gets(plain);
printf("entertheencryptionkey:");
gets(key);
kl=strlen(key);
for(i=0;plain[i]!='\0';i++)
cipher[i]=plain[i]^kl;
cipher[i]='\0';
printf("encryptedtextis:");
puts(cipher);
printf("Decryptedtextis:");
for(i=0;cipher[i]!='\0';i++)
plain[i]=cipher[i]^kl;
plain[i]<'\0';
puts(plain);
}
```

Output:

Enter the plain text:abc

Enter the key:3

Encrypted text is:def

Decryption

Enter encrypted data: def

Enter the key:3

Decrypted text:abc

Viva Questions:

1. What is encryption?

Answer:

Encryption is the process of converting plain text into unreadable text (cipher text) to protect data.

2. What is decryption?

Answer:

Decryption is the process of converting cipher text back into the original plain text.

3. What is a key in encryption?

Answer:

A key is a secret value used in an encryption algorithm to convert plain text into cipher text and vice versa.

4. What are the two main types of encryption?

Answer:

- Symmetric encryption (same key for encryption and decryption)
- Asymmetric encryption (different public and private keys)

5. Give an example of an encryption algorithm.

Answer:

Examples include AES, RSA, and DES

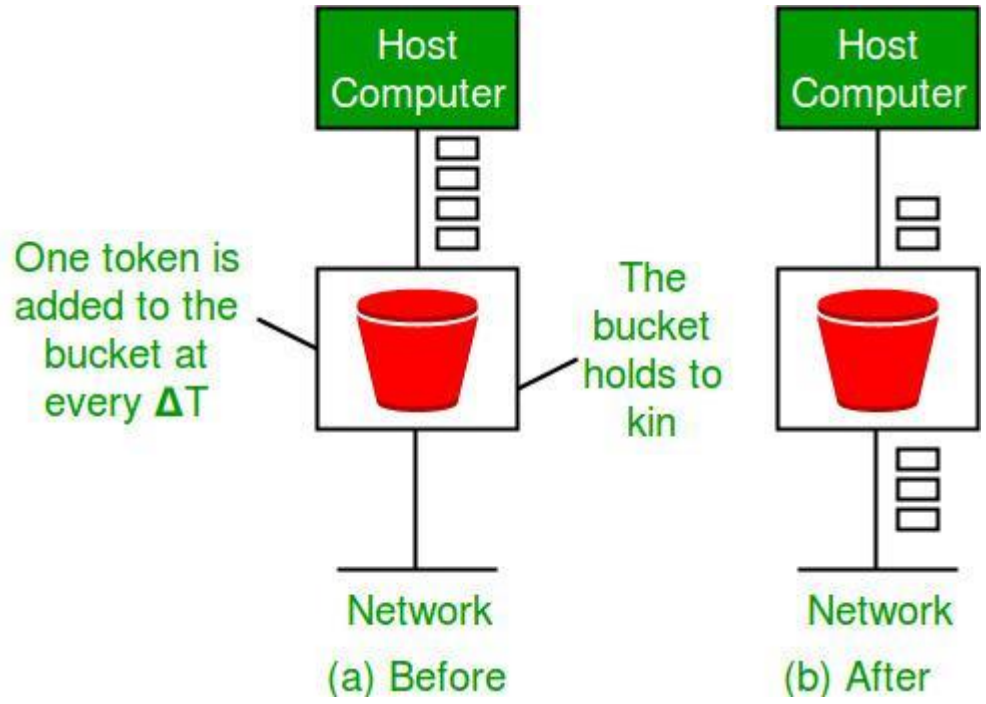
8. Write a program for congestion control using Leaky bucket algorithm.

Aim:

To write a program to implement congestion control using the Leaky Bucket Algorithm

Descriptions:

The Leaky Bucket Algorithm is a traffic shaping and congestion control mechanism used in computer networks to regulate the flow of data packets. It works like a bucket with a small hole at the bottom, where incoming packets are added to the bucket and transmitted at a constant rate. If the incoming data rate exceeds the bucket capacity, excess packets are discarded, preventing network congestion. This ensures a smooth and controlled output flow regardless of bursty input traffic. The algorithm helps in maintaining a steady transmission rate and avoids sudden overload in the network. It is widely used in networking systems to manage bandwidth and improve overall performance.



Program:

```
#include<stdio.h>

#include<stdlib.h>

#include<math.h>

#include<string.h>

intx,y,t,n,i,j,flag;

longinte[2000],d[2000],temp[2000],m[2000],en[2000];

charmsg[500];

longintprime(longint);

voidencryption_key();

doublecd(longint);

voidencrypt();

voiddecrypt();

intmain()

{

printf("\nenterthefirstprimeno.:");

scanf("%d",&x);

flag=prime(x);

if(flag==0)

{
```

```
printf("\nInvalidinput\n");
exit(0);
}
printf("\nentersecondprimeno.:\n");
scanf("%d",&y);
flag=prime(y);
if(flag==0||x==y)
{
printf("\ninvalidinput\n");
exit(0);
}
printf("\nentermsgorstringtoencrypt\n");
scanf("%s",msg);
for(i=0;msg[i]!='\0';i++)
m[i]=msg[i];
n=x*y;
t=(x-1)*(y-1);
encryption_key();
printf("\npossiblevaluesofeanddare\n");
for(i=0;i<j-1;i++)
```

```
printf("\n%ld\t%ld",e[i],d[i]);
encrypt();
decrypt();
return0;
}
longintprime(longintpr)
{
inti;
j=sqrt(pr);
for(i=2;i<=j;i++)
{
if(pr%i==0)
return0;
}
return1;
}
voidencryption_key()
{
intk;
k=0;
```

```
for(i=2;i<t;i++)
{
if(t%i==0)
continue;
flag=prime(i);
if(flag==1&&i!=x&&i!=y)
{
e[k]=i;
flag=cd(e[k]);
if(flag>0)
{
d[k]=fl
ag;
k++;
}
if(k==99)
break;
}
}
}
```

```
doublecd(longinta)
{
longintk=1;
while(1)
{
k=k+t;
if(k%a==0)
return(k/a);
}
}
voidencrypt()
{
longintpt,
i=0;
len=strl
ct,key=e[
en(msg);
while(i
!=len)
{
```

```
pt=m[i];
pt=pt-96;
k=1;
for(j
=0;j<key;
{
k=k*pt;
k=k%n;
}
temp[i]=k;
ct=k+96;
en[i]
=ct;
i++;
}
0],k,
len;
j++)
en[i]='\0';
printf("\nencryptedmessageis\n");
```

```
for(i=0;en[i]!='\0';i++)
printf("%c",en[i]);
}
voiddecrypt()
{
longintpt,ct,key=d[0],k;
i=0;
while(en[i]!='\0')
{
ct=temp[i];
k=1;
for(j=0;j<key;j++)
{
k=k*ct;
k=k%n;
}
pt=k+96;
m[i]=pt;
i++;
}
```

```
m[i]='\0';

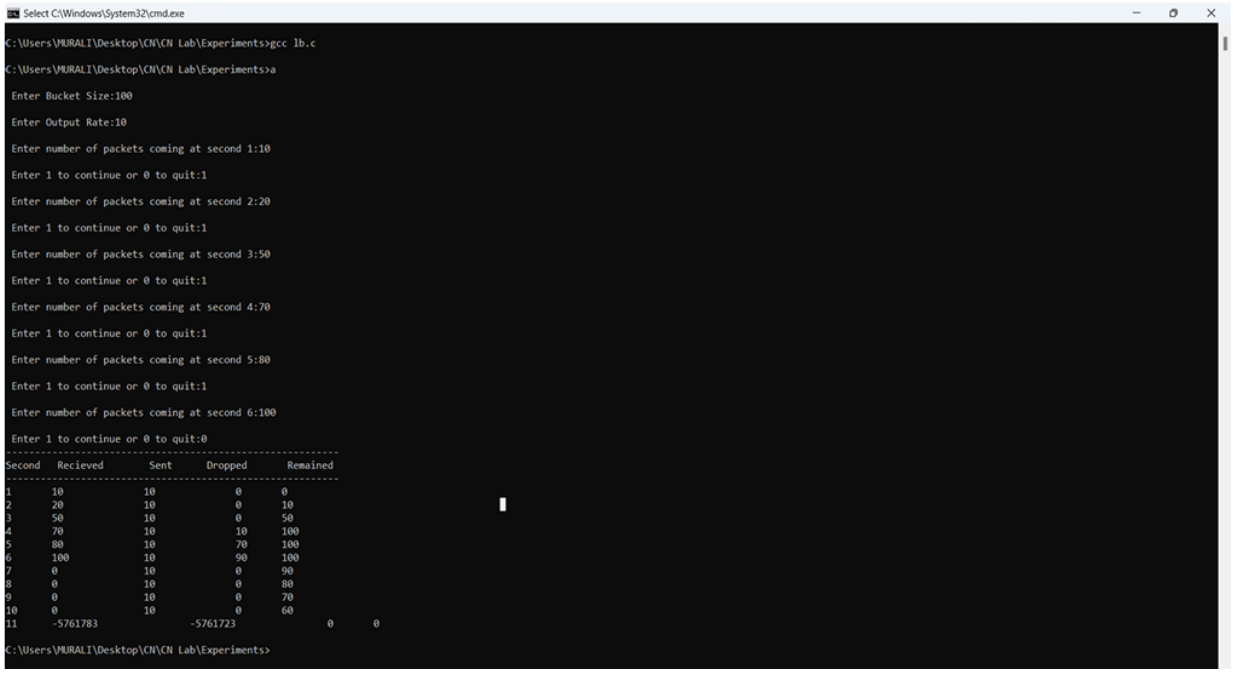
printf("\n\ndecrpytedmessage\n");

for(i=0;m[i]!='\0';i++)

printf("%c",m[i]);

printf("\n");
```

Output:



Viva Questions:

1. What is the Leaky Bucket algorithm?

Answer:

Leaky Bucket is a congestion control algorithm that controls the rate of data transmission by sending data at a fixed rate, similar to water leaking from a bucket.

2. What is the main purpose of the Leaky Bucket algorithm?

Answer:

Its main purpose is to prevent network congestion by smoothing bursty traffic into a steady output rate.

3. What happens if the bucket becomes full?

Answer:

If the bucket is full, incoming packets are discarded.

4. What are the main parameters of the Leaky Bucket algorithm?

Answer:

- Bucket size (capacity)
- Output rate (leak rate)

5. What is the difference between Leaky Bucket and Token Bucket?

Answer:

Leaky Bucket sends data at a fixed rate, while Token Bucket allows bursty transmission if tokens are available**.**

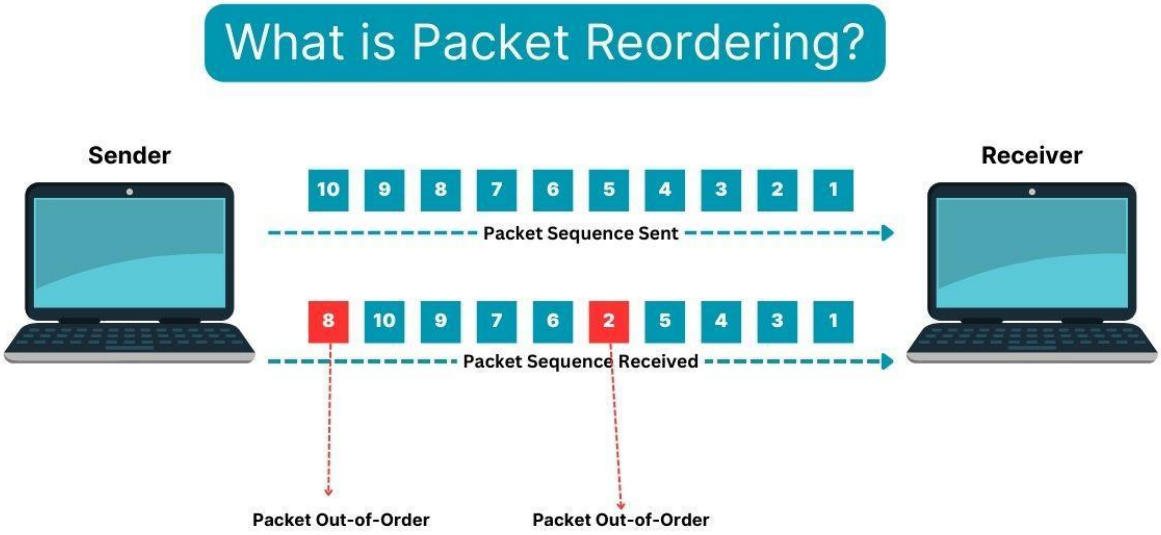
9. Write a program for frame sorting techniques used in buffers

Aim:

To write a program to implement frame sorting techniques used in buffers.

Descriptions:

Frame sorting in buffers is an important concept in computer networks, especially at the Data Link Layer, where frames may arrive out of order due to transmission delays or network issues. Buffering is used at the receiver side to temporarily store incoming frames until they can be processed in the correct sequence. Frame sorting techniques arrange these frames based on their sequence numbers before delivering them to higher layers. This ensures proper data reconstruction and avoids errors caused by misordered frames. Sorting methods such as simple comparison-based techniques can be used to reorder frames efficiently. This experiment demonstrates how buffers manage and organize frames to maintain data integrity and reliable communication.



Program:

```
#include <stdio.h>
int main() {
    int n, i, j, temp;
    int seq[50];
    printf("Enter number of frames: ");
    scanf("%d", &n);
    printf("Enter the sequence numbers (out of order):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &seq[i]);
    }
    printf("\nBefore sorting (in buffer):\n");
    for (i = 0; i < n; i++) {
        printf("%d ", seq[i]);
    }
        for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (seq[j] > seq[j + 1]) {
                temp = seq[j];
                seq[j] = seq[j + 1];
                seq[j + 1] = temp;
    printf("\n\nAfter sorting (frames in order):\n");
    for (i = 0; i < n; i++) {
        printf("%d ", seq[i]); }
     printf("\n");
    return 0;
}
```

Output:

Enter number of frames: 5

Enter the sequence numbers (out of order):

4 1 3 5 2

Before sorting (in buffer):

4 1 3 5 2

After sorting (frames in order):

1 2 3 4 5

Viva Questions:

1. What is a buffer in networking?

Answer:

A buffer is temporary memory used to store data frames before they are processed or transmitted.

2. Why do we need frame sorting in buffers?

Answer:

Frame sorting ensures that frames are delivered to the receiver in the correct order, even if they arrive out of order.

3. Name some frame sorting techniques.

Answer:

- FIFO (First-In-First-Out)
- Priority Sorting
- Sequence Number Sorting

4. What is FIFO frame sorting?

Answer:

FIFO delivers frames in the order they arrive in the buffer, regardless of priority or sequence number.

5. How does sequence number sorting work?

Answer:

Each frame has a sequence number, and frames are sorted in the buffer based on these numbers before delivery

10. How to run N map scan.

Aim

To implement a **Client-Server application using Socket programming** in C language.

Descriptions:

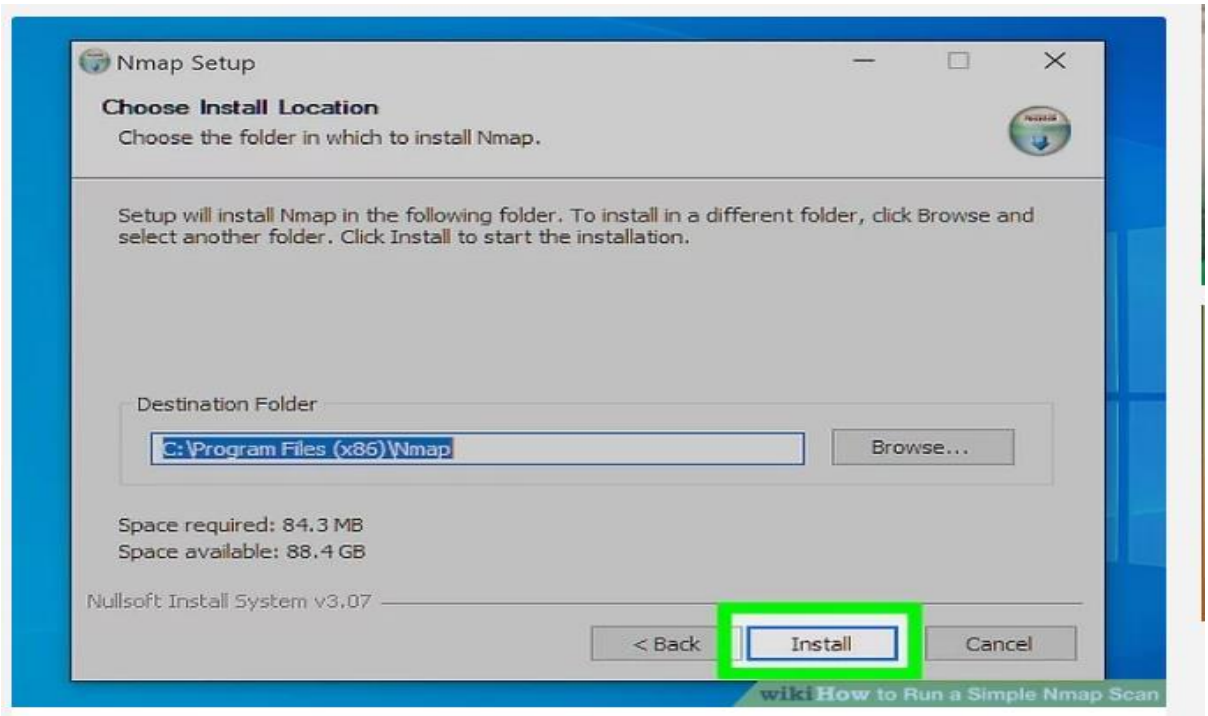
Socket programming enables communication between two systems over a network using a client-server model. The server creates a socket, binds it to a port, listens for incoming connections, and accepts client requests. The client creates a socket and connects to the server using its IP address and port number. Data is exchanged using send() and read() functions. This experiment demonstrates reliable communication using TCP protocol. It is widely used in real-world applications like web servers, chat applications, and file transfer systems.

Program

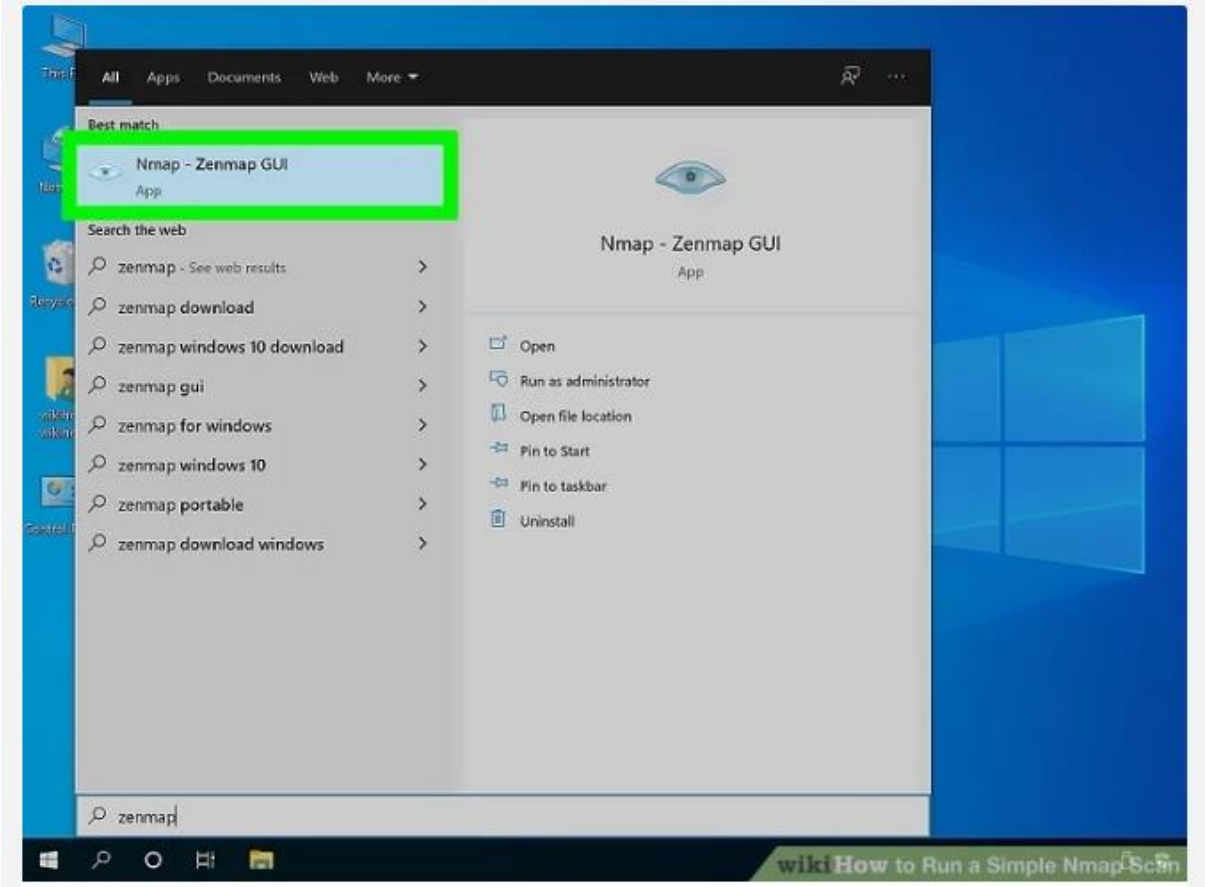
Step 1: Download the Nmap installer. This can be found for free on the developer's website. It is highly recommended that you download directly from the developer to avoid any potential viruses or fake files. Downloading the Nmap installer includes Zenmap, the graphical interface for Nmap which makes it easy for newcomers to perform scans without having to learn command lines.

- The Zenmap program is available for Windows, Linux, and Mac OS X. You can find the installation files for all operating systems on the Nmap website.

Step 2: Install Nmap. Run the installer once it is finished downloading. You will be asked which components you would like to install. To get the full benefit of Nmap, keep all of these checked. Nmap will not install any adware or spyware.

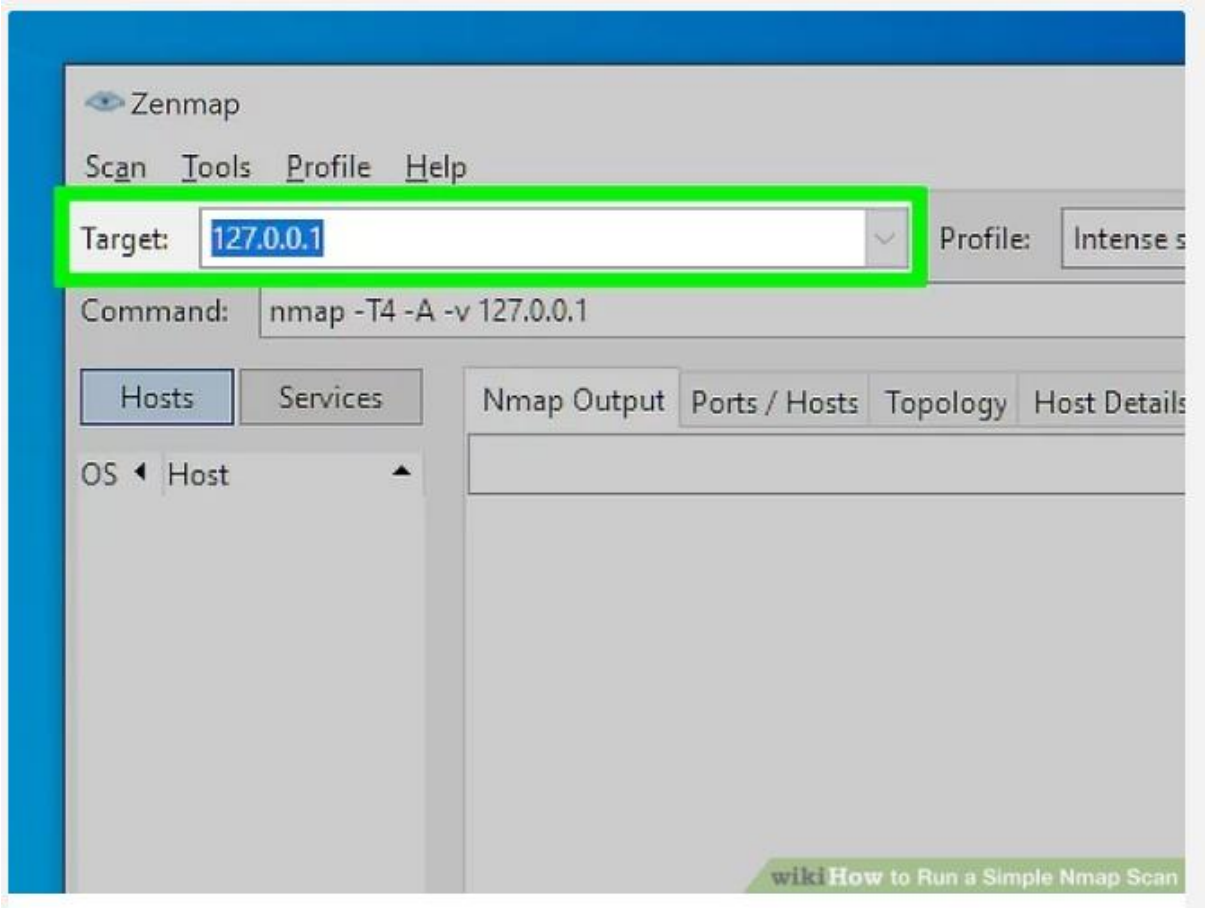


Step 3:Run the "Nmap – Zenmap" GUI program. If you left your settings at default during installation, you should be able to see an icon for it on your desktop. If not, look in your Start menu. Opening Zenmap will start the program.



Step 4:Enter the target for your scan. The Zenmap program makes scanning a fairly simple process. The first step to running a scan is choosing your target. You can enter a domain (example.com), an IP address (127.0.0.1), a network (192.168.1.0/24), or a combination of those.

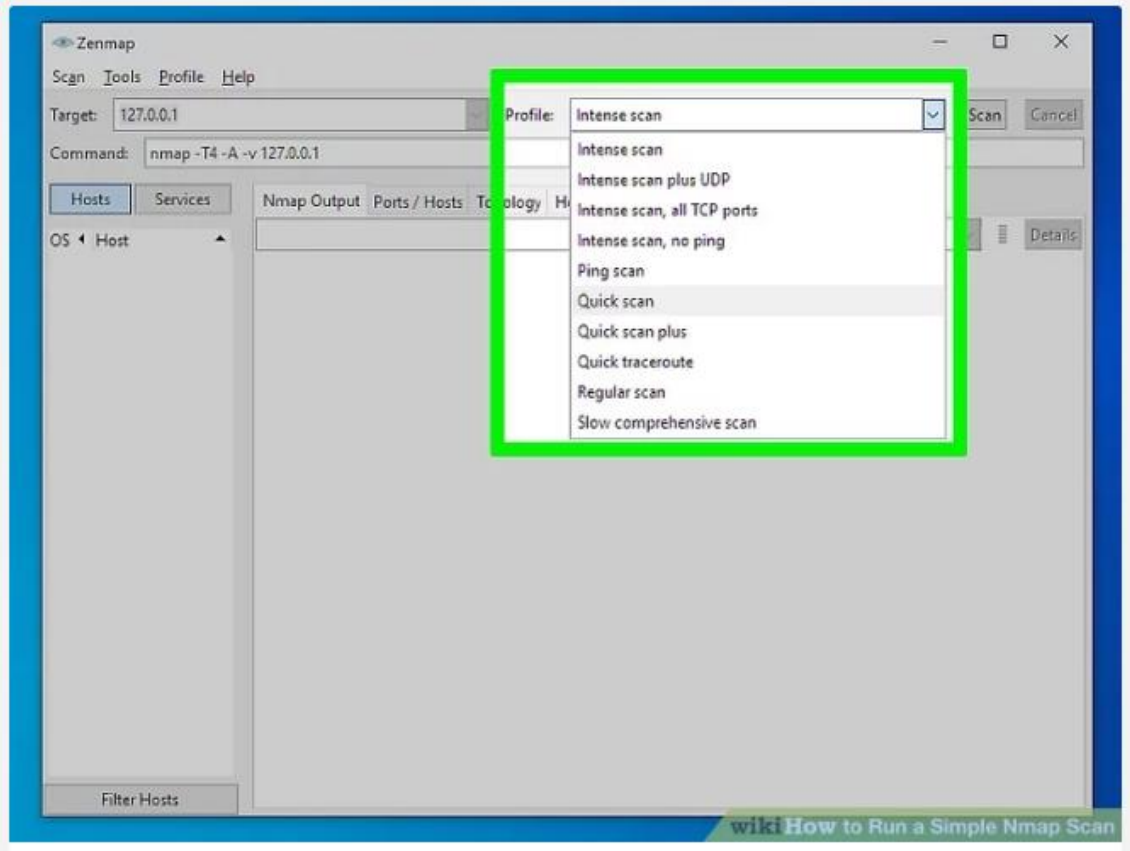
- Depending on the intensity and target of your scan, running a Nmap scan may be against the terms of your internet service provider and may land you in hot water. Always check your local laws and your ISP contract before performing Nmap scans on targets other than your network.



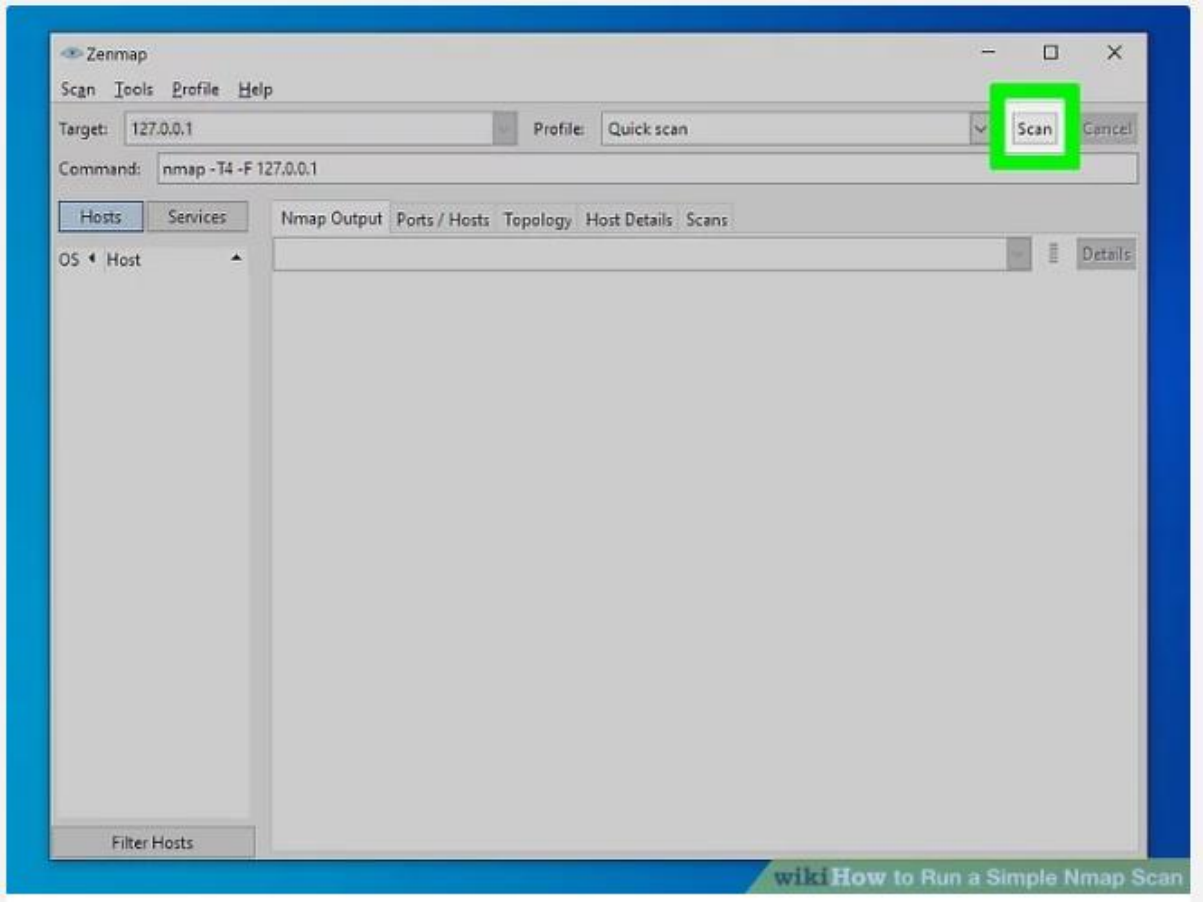
Step 5:Choose your Profile. Profiles are preset groupings of modifiers that change what is scanned. The profiles allow you to quickly select different types of scans without having to type in the modifiers on the command line. Choose the profile that best fits your needs:[1]

- **Intense scan** - A comprehensive scan. Contains Operating System (OS) detection, version detection, script scanning, and traceroute, and has aggressive scan timing. This is considered an intrusive scan.
- **Ping scan** - This scan simply detects if the targets are online, it does not scan any ports.
- **Quick scan** - This is quicker than a regular scan due to aggressive timing and only scanning select ports.

- **Regular scan** - This is the standard Nmap scan without any modifiers. It will return ping and return open ports on the target.



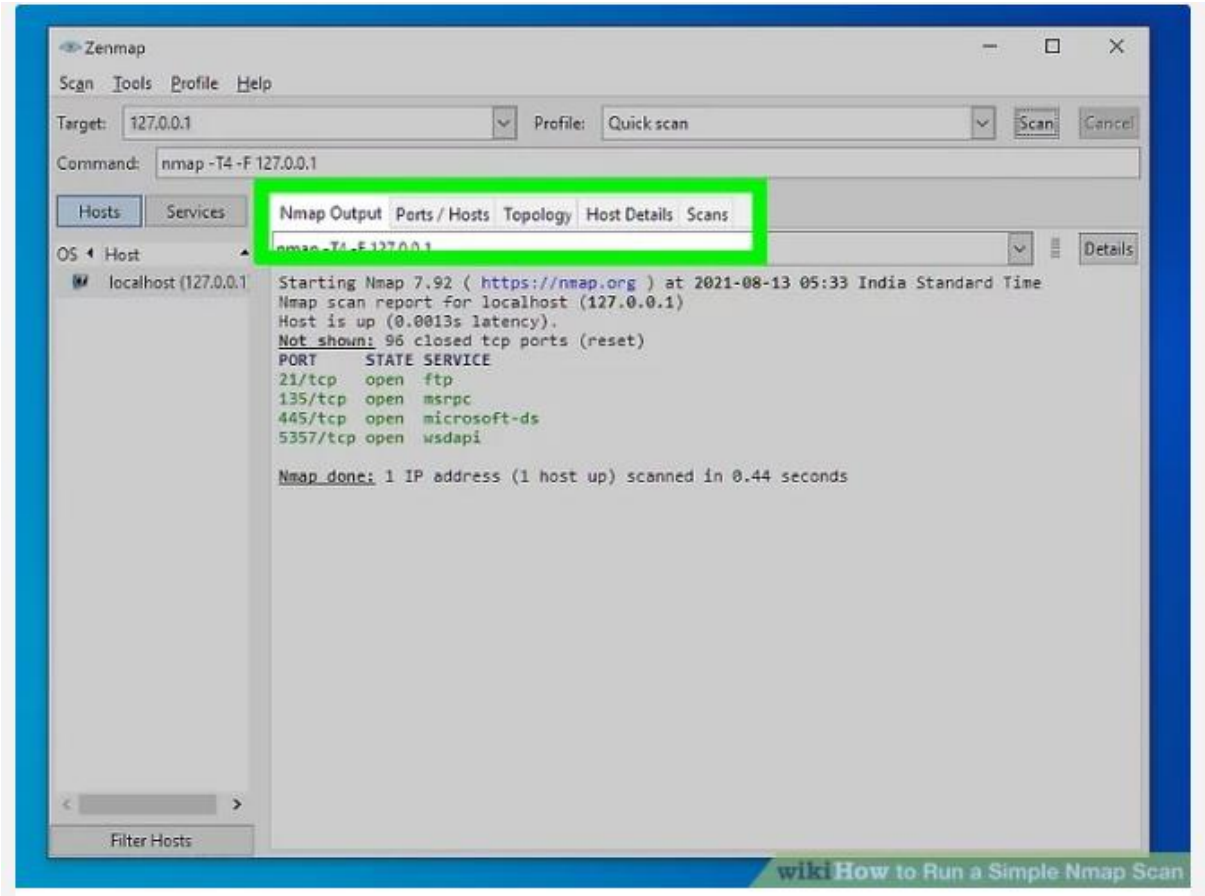
2. **Step 6:Click Scan to start scanning.** The active results of the scan will be displayed in the Nmap Output tab. The time the scan takes will depend on the scan profile you chose, the physical distance to the target, and the target's network configuration.



Step 7:Read your results. Once the scan is finished, you'll see the message "Nmap done" at the bottom of the Nmap Output tab. You can now check your results, depending on the type of scan you performed. All of the results will be listed in the main Nmap Output tab, but you can use the other tabs to get a better look at specific data.[2]

- **Ports/Hosts** - This tab will show the results of your port scan, including the services for those ports.
- **Topology** - This shows the traceroute for the scan you performed. You can see how many hops your data goes through to reach the target.

- **Host Details** - This shows a summary of your target learned through scans, such as the number of ports, IP addresses, hostnames, operating systems, and more.
- **Scans** - This tab stores the commands of your previously-run scans. This allows you to quickly re-scan with a specific set of parameters.



Viva Questions:

1. What is Nmap?

Answer:

Nmap (Network Mapper) is a network scanning tool used to discover hosts, open ports, and services running on a network.

2. What is the basic command to scan a target using Nmap?

Answer:

nmap <target_ip>

Example: nmap 192.168.1.1

3. What does the -sV option do in Nmap?

Answer:

The -sV option detects the version of services running on open ports.

4. What is the purpose of the -p option in Nmap?

Answer:

The -p option is used to specify particular ports to scan.

5. Is it legal to scan any network using Nmap?

Answer:

No. You should only scan systems you own or have permission to test, as unauthorized scanning may be illegal.

11.Operating System Detection using N map.

Aim:

To analyze network packets using Wireshark tool.

Description:

The most basic version of the Nmap command for Operating System detection is:

1 nmap -O <target>

While it's not always easy to tell on screen, the flag on this command is -O for Operating System. Not to be confused with the lowercase -o which is used for Output, or the number Zero. <target> should be switched with whatever you're trying to scan. That could be either the hostname of the target such as scanme.nmap.org, or the IP address of the target such as 45.33.32.156.

```
root@ubuntu-s-1vcpu-1gb-lon1-01:~# nmap -O scanme.nmap.org
Starting Nmap 7.80 ( https://nmap.org ) at 2022-09-22 14:05 UTC
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.14s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 closed ports
PORT      STATE    SERVICE
22/tcp    open     ssh
80/tcp    filtered http
9929/tcp  open     nping-echo
31337/tcp open     Elite
Aggressive OS guesses: Linux 2.6.32 - 3.13 (96%), Linux 2.6.22 - 2.6.36 (95%), Linux 3.10 - 4.11 (95%),
Linux 3.10 (94%), Linux 2.6.32 (94%), Linux 3.2 - 4.9 (94%), Linux 2.6.32 - 3.10 (93%), HP P2000 G3 NAS
device (93%), Linux 2.6.18 (93%), Linux 3.16 - 4.6 (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 16 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.73 seconds
```

Nmap OS detection in action

Nmap os detection confidence levels

As you'll see in the above scan screenshot, Nmap doesn't always know for sure the exact operating system in use by a target device. In those cases, it makes predictions with confidence levels shown as percentages, e.g. Linux 2.6.32 – 3.13 (96%). Because of how Nmap os detection

works unless the device replies with the exact OS and version number this can be the best you can get. The more information Nmap can gather, the more accurate the prediction will be.

False positive reduction with Nmap os detection

Because of how os detection works, sometimes a basic -O scan will have many different high-confidence guesses and it's possible that none of them are completely accurate. A good trick to remove the noise and confirm the underlying operating system is to run an aggressive Nmap scan with both OS and application version detection:

1 nmap -A -T4 <target>

The -A flag includes -O for Operating System detection and upgrades it to Application version detection. The -T4 flag is for aggressive timing (T0 to T5, least to most aggressive). Application version detection can take quite a while with default timing so T4 helps to speed things up a little..

```
root@ubuntu-s-1vcpu-1gb-lon1-01:~# nmap -A -T4 scanme.nmap.org
Starting Nmap 7.80 ( https://nmap.org ) at 2022-09-22 14:41 UTC
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.14s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 closed ports
PORT      STATE SERVICE    VERSION
22/tcp    open  ssh        OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   1024 ac:00:a0:1a:82:ff:cc:55:99:dc:67:2b:34:97:6b:75 (DSA)
|   2048 20:3d:2d:44:62:2a:b0:5a:9d:b5:b3:05:14:c2:a6:b2 (RSA)
|   256 96:02:bb:5e:57:54:1c:4e:45:2f:56:4c:4a:24:b2:57 (ECDSA)
|_  256 33:fa:91:0f:e0:e1:7b:1f:6d:05:a2:b0:f1:54:41:56 (ED25519)
80/tcp    open  http       Apache httpd 2.4.7 ((Ubuntu))
|_http-server-header: Apache/2.4.7 (Ubuntu)
|_http-title: Go ahead and ScanMe!
9929/tcp  open  nping-echo Nping echo
31337/tcp open  tcpwrapped
Aggressive OS guesses: Linux 2.6.32 - 3.13 (96%), Linux 2.6.22 - 2.6.36 (95%), Linux 3.10 - 4.11 (94%),
Linux 3.2 - 4.9 (94%), Linux 2.6.32 - 3.10 (93%), HP P2000 G3 NAS device (93%), Linux 2.6.18 (93%), Linu
x 2.6.32 (93%), Linux 3.10 (93%), Linux 3.16 - 4.6 (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 10 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

nmapos and version detection

The screenshot above shows the result of an -A -T4 scan against the same target as before. Unfortunately, the Aggressive OS guesses are still the same. Thankfully, in this example, we can see that the OpenSSH version states 2ubuntu2.13 which indicates that the server is running on Ubuntu operating system. A quick Google for "ubuntu OpenSSH versions" indicates that the server is most likely to be running on ubuntu 14.04, assuming the server is fully patched.

Upgrading OpenSSH on Ubuntu LTS

If you receive a PCI compliance warning telling you to update to the most recent version of OpenSSH, your PCI scanner is most likely showing a false positive.

OpenSSH 6.6 is the most recent version on Ubuntu 14.04.

OpenSSH 7.2 is the most recent version on Ubuntu 16.04.

OpenSSH 7.6 is the most recent version on Ubuntu 18.04.

OpenSSH 8.2 is the most recent version on Ubuntu 20.04.

OpenSSH 8.9 is the most recent version on Ubuntu 22.04.

Nmap os detection: ubuntu

Basic firewall evasion for Nmap os detection scans

Some servers and firewalls are configured to block ICMP ping scans and Nmap uses ping to test if a server is alive before trying to port scan it. To bypass this simply disable the default ping check by using the command:

```
1    nmap -O -Pn<target>
```

The -Pn flag disables the ping check: **n**o **P**ing aka **P**ing:**n**o. Nmap then assumes that the host is up and performs a port scan and os detection scan. This can be combined with other scan types such as:

```
1    nmap -Pn -A -T4 <target>
```

The result is a scan that skips ping checks, tests for operating system versions and application versions, and runs aggressively with probing messages sent rapidly for faster results.

These scans are easily detected and blocked by modern, advanced, and well-configured firewalls. Fortunately, most devices aren't protected by modern, advanced firewalls. Those that are, more often than not, are poorly configured

Viva Questions:

1. What is Wireshark?

Answer:
Wireshark is a network protocol analyzer used to capture and analyze packets in real-time. It helps in monitoring, troubleshooting, and analyzing network communication.

2. What is Nmap used for?

Answer:
Nmap (Network Mapper) is used for network discovery and security auditing. It helps in scanning open ports, detecting services, and identifying operating systems of target machines.

3. What is NS2 Simulator?

Answer:
NS2 is a network simulation tool used to model and analyze network protocols, traffic, congestion, and performance. It uses TCL scripting to simulate network scenarios.

4. What is packet drop and why does it occur?

Answer:
Packet drop occurs when network devices fail to deliver packets due to congestion, buffer overflow, or transmission errors. It affects network performance and reliability.

5. Difference between TCP and UDP?

Answer:
TCP is a reliable, connection-oriented protocol that ensures data delivery with error checking, while UDP is a connectionless protocol that is faster but does not guarantee delivery.

12. Study of basic Network configuration commands and utilities to debug the network issues.

Description:

Here are some basic network configuration commands and utilities that are essential for configuring networks and troubleshooting network issues:

1. IP Configuration and Information

- **ipconfig (Windows)**: Displays IP configuration details such as IP address, subnet mask, and default gateway for all network adapters.

ipconfig /all # For detailed information
ipconfig /release # Release current IP address
ipconfig /renew # Renew IP address

```
Windows IP Configuration


Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . . : 2401:4900:3688:8c30:4d9e:c61:5b3e:4120
   Temporary IPv6 Address. . . . . . . : 2401:4900:3688:8c30:21f1:8f11:52ec:ce2
   Link-local IPv6 Address . . . . . . : fe80::27f1:2bbd:2e31:9fd3%11
   IPv4 Address. . . . . . . . . . . . : 192.168.199.184
   Subnet Mask . . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . . : fe80::6052:b7ff:fe3b:6464%11
                                         192.168.199.27
```

- **ifconfig (Linux/macOS)**: Used to display or configure a network interface.

ifconfig # View current network settings
ifconfig eth0 down # Disable a network interface
ifconfig eth0 up # Enable a network interface

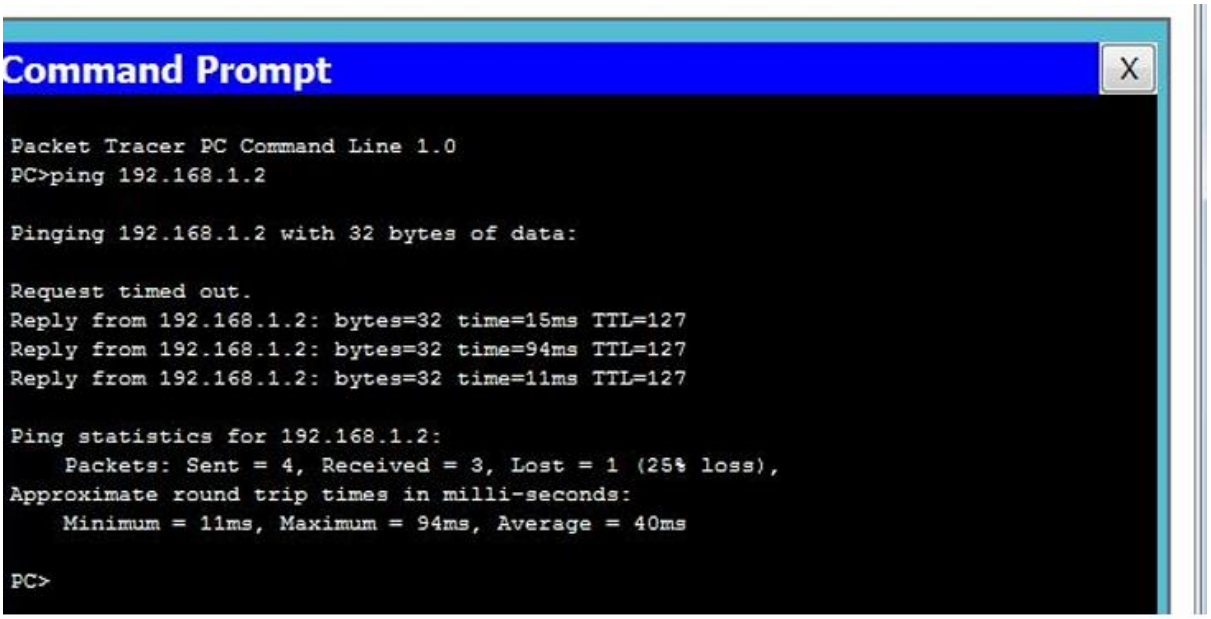
- **ip (Linux)**: A more powerful command to replace ifconfig.

ipa # Show all interfaces and IP addresses
ip link set eth0 up # Enable a network interface
ip link set eth0 down # Disable a network interface

2. Check Network Connectivity

- **ping**: Sends ICMP echo requests to a host to check connectivity.

ping google.com # Check connectivity to Google
ping -c 4 8.8.8.8 # Ping a specific IP address 4 times (Linux/macOS)



- **traceroute (Linux/macOS) / tracert (Windows)**: Traces the path to a remote host.

traceroute google.com # Linux/macOS
tracertgoogle.com # Windows

3. View and Manipulate Routing Tables

- **route**: Display or modify the IP routing table.

route print # View routing table (Windows)
route -n # View routing table (Linux)

- **ip route** (Linux): A modern command to show and manipulate routing tables

ip route show # Display current routing table
ip route add default via 192.168.1.1 # Add a default gateway.

4. DNS Utilities

- **nslookup**: Query Internet name servers interactively.

nslookupgoogle.com # Look up DNS information for a domain

- **dig** (Linux/macOS): Provides detailed DNS query results.

dig google.com # Query domain name information
dig google.com +trace # Trace the path of DNS resolution

5. Network Statistics

- **netstat**: Display network connections, routing tables, and more.

netstat -a # Show all active connections
netstat -r # Show routing table
netstat -an | grep LISTEN # Show listening ports (Linux)

6. Diagnosing ARP Issues

- **arp**: View or manipulate the ARP cache.

arp -a # Display ARP table (Windows)
arp -n # Display ARP table without DNS resolution (Linux)
arp -d 192.168.1.1 # Delete a specific ARP entry

7. Bandwidth and Packet Analysis

- **tcpdump** (Linux/macOS): Capture and analyze network packets.

tcpdump -i eth0 # Capture packets on eth0 interface
tcpdump -c 10 -i eth0 # Capture 10 packets

- **Wireshark**: A graphical tool for network protocol analysis.

8. Network Service Testing

- **telnet**: Test connectivity to a specific port.

telnet google.com 80 # Test HTTP connection to google.com

- **curl**: Command-line tool for transferring data with URLs.

curl -I http://example.com # Fetch HTTP headers of a URL
curl http://example.com # Download a webpage

- **netcat (nc)**: Network utility for reading from and writing to network connections.

nc -zv 192.168.1.1 22 # Check if port 22 is open on a remote server

These commands provide a comprehensive toolkit for diagnosing and resolving network issues on Windows, Linux, and macOS.

9. ping

- **Usage**: Test connectivity between your system and another host.
- **Example**:

bash
Copy code
ping google.com

10. tracert (Windows) / traceroute (Linux)

- **Usage**: Trace the path packets take to reach a network host.
- **Example**:
  - **Windows**:

tracert google.com

```
Tracing route to google.com [2404:6800:4007:81b::200e]
over a maximum of 30 hops:

  1     4 ms     1 ms     1 ms  2401:4900:3688:8c30::c0
  2     *        *        *     Request timed out.
  3    71 ms    25 ms    48 ms  2401:4900:d0:4001::205
  4    54 ms    34 ms    38 ms  2401:4900:c0:1::99
  5    55 ms    37 ms    47 ms  2404:a800:3a00:2::39
  6    64 ms    57 ms    50 ms  2404:a800::92
  7    76 ms    51 ms    42 ms  2001:4860:1:1::d2e
  8    61 ms    48 ms    48 ms  2404:6800:8134::1
  9    69 ms    48 ms    37 ms  2001:4860:0:1::55ca
 10    59 ms    47 ms    47 ms  2001:4860:0:1::17d6
 11    64 ms    56 ms    56 ms  2001:4860:0:1::8825
 12    53 ms    46 ms    59 ms  2001:4860:0:1::5667
 13    62 ms    38 ms    46 ms  maa05s20-in-x0e.1e100.net [2404:6800:4007:81b::200e]

Trace complete.
```

- **Linux**:

traceroute google.com

11**. nslookup**

- **Usage**: Query Domain Name System (DNS) information.
- **Example**:

nslookup google.com

```
Server:  UnKnown
Address:  192.168.199.27

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4007:825::200e
          142.250.195.142
```

12 **. netstat**

- **Usage**: Display network connections, routing tables, and other network interface statistics.
- **Example**:

netstat -a

13. route

- **Usage**: Display or modify the routing table.
- **Example**:

route print

```
Interface List
 16...90 65 84 a0 aa f6 ......Microsoft Wi-Fi Direct Virtual Adapter
  3...92 65 84 a0 aa f5 ......Microsoft Wi-Fi Direct Virtual Adapter #2
 11...90 65 84 a0 aa f5 ......Intel(R) Wi-Fi 6 AX201 160MHz
  9...90 65 84 a0 aa f9 ......Bluetooth Device (Personal Area Network)
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0   192.168.199.27  192.168.199.184     55
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
    192.168.199.0    255.255.255.0         On-link   192.168.199.184    311
  192.168.199.184  255.255.255.255         On-link   192.168.199.184    311
  192.168.199.255  255.255.255.255         On-link   192.168.199.184    311
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link   192.168.199.184    311
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link   192.168.199.184    311
===========================================================================
Persistent Routes:
  None

IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
 11     71 ::/0                     fe80::6052:b7ff:fe3b:6464
  1    331 ::1/128                  On-link
 11     71 2401:4900:3688:8c30::/64 On-link
 11    311 2401:4900:3688:8c30:21f1:8f11:52ec:ce2/128
                                    On-link
 11    311 2401:4900:3688:8c30:4d9e:c61:5b3e:4120/128
                                    On-link
 11    311 fe80::/64                On-link
 11    311 fe80::27f1:2bbd:2e31:9fd3/128
                                    On-link
  1    331 ff00::/8                 On-link
 11    311 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

14.dig (Linux)

- **Usage**: Perform DNS lookups and provide detailed DNS information.
- **Example**:

dig google.com

15.arp

- **Usage**: Display or modify the Address Resolution Protocol (ARP) table.
- **Example**:

arp -a

```
Interface: 192.168.199.184 --- 0xb
  Internet Address      Physical Address      Type
  192.168.199.27        62-52-b7-3b-64-64     dynamic
  192.168.199.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

16.ip (Linux)

- **Usage**: Newer and more powerful command than ifconfig to display and manipulate routing, devices, and tunnels.
- **Example**

ipaddr show

17. telnet

- **Usage**: Test the connection to a remote host on a specific port.
- **Example**:

telnet google.com 80

These commands and tools will help diagnose network connectivity, analyze network traffic, resolve DNS issues, and inspect routing details.

Viva Questions

1. **What is the purpose of the `ipconfig` (Windows) or `ifconfig` / `ip` (Linux) command?**
   Used to display and configure IP address, subnet mask, and gateway of a system.

2. **How does the `ping` command help in network troubleshooting?**
   It checks connectivity between two devices and measures response time using ICMP packets.

3. **What is the use of the `tracert` (Windows) / `traceroute` (Linux) command?**
   It shows the path (route) taken by packets to reach the destination and identifies delays or failures.

4. **Explain the function of the `netstat` command.**
   Displays active network connections, listening ports, and protocol statistics.

5. **What is DNS and which command is used to test it?**
   DNS (Domain Name System) translates domain names into IP addresses.
   Commands like `nslookup` or `dig` are used to test DNS resolution.